```
package com.chenpp.graph.admin;

import org.mybatis.spring.annotation.MapperScan;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.scheduling.annotation.EnableAsync;
import org.springframework.context.annotation.EnableAspectJAutoProxy;

@SpringBootApplication
@ComponentScan(basePackages = "com.chenpp.graph")
@MapperScan("com.chenpp.graph.admin.mapper")
@EnableAsync
@EnableAspectJAutoProxy
public class GraphAdminApplication {
    public static void main(String[] args) {
        SpringApplication.run(GraphAdminApplication.class, args);
    }
}

package com.chenpp.graph.admin.controller;

import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.chenpp.graph.admin.model.GraphConnection;
import com.chenpp.graph.admin.model.Result;
import com.chenpp.graph.admin.service.GraphConnectionService;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.time.LocalDateTime;

/**
 * 图数据库连接管理 API

 */
@Slf4j
@RestController
@RequestMapping("/api/connections")
public class GraphConnectionController {

    @Autowired
    private GraphConnectionService connectionService;

    @GetMapping
    public Result<Page<GraphConnection>> getConnections(
            @RequestParam(defaultValue = "1") Integer page,
            @RequestParam(defaultValue = "10") Integer pageSize,
            @RequestParam(required = false) String keyword,
            @RequestParam(required = false) String graphType) {

        Page<GraphConnection> pageObj = new Page<>(page, pageSize);
```

```
        Page<GraphConnection> result = connectionService.queryConnections(pageObj, keyword, graphType);
        return Result.success(result);
    }

    @PostMapping
    public Result<Boolean> createConnection(@RequestBody GraphConnection connection) {
        boolean success = connectionService.save(connection);
        return Result.success(success);
    }

    @PutMapping("/{id}")
    public Result<Boolean> updateConnection(@PathVariable Long id, @RequestBody GraphConnection connection) {
        connection.setId(id);
        connection.setUpdateTime(LocalDateTime.now());
        boolean success = connectionService.updateById(connection);
        return Result.success(success);
    }

    @DeleteMapping("/{id}")
    public Result<Boolean> deleteConnection(@PathVariable Long id) {
        boolean success = connectionService.removeById(id);
        return Result.success(success);
    }

    @PostMapping("/{id}/test")
    public Result<Boolean> testConnection(@PathVariable Long id) {
        boolean success = connectionService.testConnection(id);
        return Result.success(success);
    }
}

package com.chenpp.graph.admin.service.impl;

import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.chenpp.graph.admin.mapper.GraphConnectionDao;
import com.chenpp.graph.admin.enums.ConnectionStatusEnum;
import com.chenpp.graph.admin.model.GraphConnection;
import com.chenpp.graph.admin.service.GraphConnectionService;
import com.chenpp.graph.admin.util.GraphClientFactory;
import com.chenpp.graph.core.GraphClient;
import com.chenpp.graph.core.model.GraphConf;
import lombok.extern.slf4j.Slf4j;
import org.apache.commons.lang3.StringUtils;
import org.springframework.stereotype.Service;

/**
 * 图数据库连接服务实现类
 */
@Slf4j
@Service
public class GraphConnectionServiceImpl extends ServiceImpl<GraphConnectionDao, GraphConnection> implements
GraphConnectionService {

    @Override
    public Page<GraphConnection> queryConnections(Page<GraphConnection> page, String keyword, String type) {
        QueryWrapper<GraphConnection> queryWrapper = new QueryWrapper<>();
        if (StringUtils.isNotBlank(keyword)) {
            queryWrapper.like("name", keyword).or().like("hosts", keyword);
```

```
        }
        if (StringUtils.isNotBlank(type)) {
            queryWrapper.eq("graph_type", type);
        }
        queryWrapper.orderByDesc("create_time");
        return this.page(page, queryWrapper);
    }


    @Override
    public boolean testConnection(Long id) {
        GraphConnection connection = this.getById(id);
        if (connection == null) {
            return false;
        }
        try {
            GraphConf graphConf = GraphClientFactory.createGraphConf(connection, connection.getGraphType());
            GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
            boolean result = graphClient.checkConnection();
            connection.setStatus(result ? ConnectionStatusEnum.CONNECTED.getCode() :
ConnectionStatusEnum.FAILED.getCode());
            return result;
        } catch (Exception e) {
            log.error("连接测试异常", e);
            connection.setStatus(ConnectionStatusEnum.FAILED.getCode());
            return false;
        } finally {
            this.updateById(connection);
        }
    }


    @Override
    public boolean save(GraphConnection entity) {
        entity.setStatus(ConnectionStatusEnum.UNCHECKED.getCode());
        return super.save(entity);
    }
}

package com.chenpp.graph.admin.mapper;

import com.baomidou.mybatisplus.core.mapper.BaseMapper;
import com.chenpp.graph.admin.model.GraphConnection;

public interface GraphConnectionDao extends BaseMapper<GraphConnection> {
}

package com.chenpp.graph.admin.model;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableField;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import com.chenpp.graph.admin.enums.GraphTypeEnum;
import lombok.Data;

import java.time.LocalDateTime;

/**
 * 图数据库连接配置信息
```

```
 */
@TableName("graph_connection")
@Data
public class GraphConnection {

    @TableId(type = IdType.AUTO)
    private Long id;

    private String name;
    @TableField(value = "graph_type")
    private String graphType;
    private String hosts;
    private Integer port;
    private String username;
    private String password;
    /**
     * 0: 未检测, 1: 通过, 2: 失败
     */
    private Integer status;
    private String description;
    private LocalDateTime createTime;
    private LocalDateTime updateTime;
    /**
     * json 参数
     */
    private String params;

    public GraphTypeEnum getGraphTypeEnum() {
        if (graphType == null) {
            return null;
        }
        return GraphTypeEnum.valueOf(graphType.toLowerCase());
    }
}


package com.chenpp.graph.admin.controller;

import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.chenpp.graph.admin.model.GraphInfo;
import com.chenpp.graph.admin.model.Result;
import com.chenpp.graph.admin.service.GraphSchemaService;
import com.chenpp.graph.admin.service.GraphService;
import com.chenpp.graph.core.exception.ErrorCode;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

/**
 * 图管理 API
```

```
 */
@Slf4j
@RestController
@RequestMapping("/api/graphs")
public class GraphController {

    @Autowired
    private GraphService graphService;

    @Autowired
    private GraphSchemaService graphSchemaService;

    @GetMapping
    public Result<Page<GraphInfo>> getGraphs(
            @RequestParam(defaultValue = "1") Integer page,
            @RequestParam(defaultValue = "10") Integer pageSize,
            @RequestParam(required = false) String keyword) {

        Page<GraphInfo> pageObj = new Page<>(page, pageSize);
        Page<GraphInfo> result = graphService.queryGraphs(pageObj, keyword);
        return Result.success(result);
    }

    @GetMapping("/list")
    public Result<Page<GraphInfo>> getGraphsByConnectionId(
            @RequestParam Long connectionId,
            @RequestParam(defaultValue = "1") Integer page,
            @RequestParam(defaultValue = "10") Integer pageSize) {

        Page<GraphInfo> pageObj = new Page<>(page, pageSize);
        Page<GraphInfo> result = graphService.queryGraphsByConnectionId(connectionId, pageObj);
        return Result.success(result);
    }

    @PostMapping
    public Result<Long> createGraph(@RequestBody GraphInfo graphInfo) {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
        String creator = authentication != null ? authentication.getName() : "unknown";
        graphInfo.setCreator(creator);
        Long graphId = graphSchemaService.createGraphInDatabase(graphInfo);
        return Result.success(graphId);
    }

    @PutMapping("/{id}")
    public Result<String> updateGraph(@PathVariable Long id, @RequestBody GraphInfo graphInfo) {
        graphInfo.setId(id);
        graphService.updateById(graphInfo);
        return Result.success("更新成功");
    }

    @DeleteMapping("/{id}")
    public Result<String> deleteGraph(
            @PathVariable Long id,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode) {
        boolean result = graphService.removeGraph(id, connectionId, graphCode);
        if (!result) {
            return Result.error(ErrorCode.GRAPH_NOT_FOUND);
        }
```

```
        return Result.success("删除成功");
    }

    @GetMapping("/{id}")
    public Result<GraphInfo> getGraph(@PathVariable Long id) {
        GraphInfo graphInfo = graphService.getById(id);
        if (graphInfo == null) {
            return Result.error(ErrorCode.GRAPH_NOT_FOUND);
        }
        return Result.success(graphInfo);
    }
}

package com.chenpp.graph.admin.service.impl;

import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.baomidou.mybatisplus.extension.plugins.pagination.Page;
import com.baomidou.mybatisplus.extension.service.impl.ServiceImpl;
import com.chenpp.graph.admin.mapper.GraphDao;
import com.chenpp.graph.admin.mapper.GraphEdgeDefDao;
import com.chenpp.graph.admin.mapper.GraphVertexDefDao;
import com.chenpp.graph.admin.model.GraphInfo;
import com.chenpp.graph.admin.model.GraphConnection;
import com.chenpp.graph.admin.model.GraphEdgeDef;
import com.chenpp.graph.admin.model.GraphPropertyDef;
import com.chenpp.graph.admin.model.GraphVertexDef;
import com.chenpp.graph.admin.service.GraphConnectionService;
import com.chenpp.graph.admin.service.GraphEdgeDefService;
import com.chenpp.graph.admin.service.GraphPropertyDefService;
import com.chenpp.graph.admin.service.GraphService;
import com.chenpp.graph.admin.service.GraphVertexDefService;
import com.chenpp.graph.admin.enums.GraphTypeEnum;
import com.chenpp.graph.admin.util.GraphClientFactory;
import com.chenpp.graph.core.GraphClient;
import com.chenpp.graph.core.GraphOperations;
import com.chenpp.graph.core.exception.BusinessException;
import com.chenpp.graph.core.model.GraphConf;
import com.chenpp.graph.core.schema.Graph;
import lombok.extern.slf4j.Slf4j;
import org.apache.commons.collections4.CollectionUtils;
import org.apache.commons.lang3.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.support.TransactionTemplate;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Set;
import java.util.concurrent.atomic.AtomicLong;
import java.util.stream.Collectors;

/**
 * 图管理服务实现类
 */
@Slf4j
@Service
public class GraphServiceImpl extends ServiceImpl<GraphDao, GraphInfo> implements GraphService {
```

```
    @Autowired
    private GraphVertexDefService vertexDefService;

    @Autowired
    private GraphEdgeDefService edgeDefService;

    @Autowired
    private GraphPropertyDefService propertyDefService;

    @Autowired
    private GraphConnectionService connectionService;

    @Autowired
    private GraphVertexDefDao graphVertexDefDao;

    @Autowired
    private GraphEdgeDefDao graphEdgeDefDao;

    @Autowired
    private TransactionTemplate transactionTemplate;


    @Override
    public Page<GraphInfo> queryGraphs(Page<GraphInfo> page, String keyword) {
        QueryWrapper<GraphInfo> queryWrapper = new QueryWrapper<>();
        if (StringUtils.isNotBlank(keyword)) {
            queryWrapper.like("name", keyword).or().like("code", keyword);
        }
        queryWrapper.orderByDesc("create_time");
        Page<GraphInfo> pageResult = this.page(page, queryWrapper);
        fillGraphCounts(pageResult.getRecords());
        fillGraphStatus(pageResult.getRecords());
        return pageResult;
    }

    @Override
    public Page<GraphInfo> queryGraphsByConnectionId(Long connectionId, Page<GraphInfo> page) {
        QueryWrapper<GraphInfo> queryWrapper = new QueryWrapper<>();
        queryWrapper.eq("connection_id", connectionId);
        queryWrapper.orderByDesc("create_time");
        Page<GraphInfo> pageResult = this.page(page, queryWrapper);

        fillGraphCounts(pageResult.getRecords());

        for (GraphInfo g : pageResult.getRecords()) {
            g.setSourceType("PLATFORM");
        }

        List<GraphInfo> allGraphInfos = new ArrayList<>(pageResult.getRecords());
        List<GraphInfo> discovered = discoverRemoteGraphs(connectionId);
        Set<String> localCodes = pageResult.getRecords().stream().map(GraphInfo::getCode).collect(Collectors.toSet());
        for (GraphInfo remote : discovered) {
            if (!localCodes.contains(remote.getCode())) {
                allGraphInfos.add(remote);
            }
        }

        fillCountsFromRemote(connectionId, allGraphInfos);

        Page<GraphInfo> resultPage = new Page<>(page.getCurrent(), page.getSize(), allGraphInfos.size());
```

```
        int ps = (int) page.getSize();
        int from = (int) ((page.getCurrent() - 1) * ps);
        int to = Math.min(from + ps, allGraphInfos.size());
        resultPage.setRecords(from < allGraphInfos.size() ? allGraphInfos.subList(from, to) : List.of());
        fillGraphStatus(resultPage.getRecords());
        return resultPage;
    }

    /**
     * 从图数据库发现已有的图，并获取节点/边类型数量
     */
    private List<GraphInfo> discoverRemoteGraphs(Long connectionId) {
        List<GraphInfo> result = new ArrayList<>();
        GraphConnection connection = connectionService.getById(connectionId);
        if (connection == null) {
            return result;
        }

        GraphInfo graphInfo = new GraphInfo();
        graphInfo.setConnectionId(connectionId);

        if (GraphTypeEnum.janus.name().equalsIgnoreCase(connection.getGraphType())) {
            log.debug("JanusGraph does not support listing existing graphs, skipping discovery");
            return result;
        }

        GraphConf graphConf = GraphClientFactory.createGraphConf(connection, "placeholder");

        try {
            GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
            GraphOperations graphOperations = graphClient.opsForGraph();
            List<Graph> remoteGraphs = graphOperations.listGraphs(graphConf);

            AtomicLong idCounter = new AtomicLong(-1);
            for (Graph rg : remoteGraphs) {
                GraphInfo g = new GraphInfo();
                Long negId = idCounter.decrementAndGet();
                g.setId(negId);
                g.setCode(rg.getCode());
                g.setName(rg.getName());
                g.setConnectionId(connectionId);
                g.setGraphType(connection.getGraphTypeEnum());
                g.setSourceType("EXISTING");
                g.setStatus(0);
                result.add(g);
            }
        } catch (Exception e) {
            log.warn("发现远程图失败", e);
        }
        return result;
    }

    @Override
    public boolean save(GraphInfo graphInfo) {
        GraphConnection connection = connectionService.getById(graphInfo.getConnectionId());
        if (connection == null) {
            throw new BusinessException("图数据库连接不存在");
        }
        graphInfo.setStatus(0);
        graphInfo.setGraphType(connection.getGraphTypeEnum());
```

```
        return super.save(graphInfo);
    }

    @Override
    public boolean removeGraph(Long graphId, Long connectionId, String graphCode) {
        GraphInfo graphInfo = null;
        GraphConnection connection = null;

        if (graphId != null && graphId > 0) {
            graphInfo = this.getById(graphId);
            if (graphInfo != null) {
                graphCode = graphInfo.getCode();
                connection = connectionService.getById(graphInfo.getConnectionId());
            }
        }

        if (graphInfo == null && connectionId != null) {
            connection = connectionService.getById(connectionId);
        }

        if (connection != null && StringUtils.isNotBlank(graphCode)) {
            GraphConf graphConf = GraphClientFactory.createGraphConf(connection, graphCode);
            GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
            GraphOperations graphOperations = graphClient.opsForGraph();
            graphOperations.dropGraph(graphConf);
            log.info("已删除图数据库中的图，code={}, connectionId={}", graphCode, connection.getId());
        }

        if (graphInfo != null && graphInfo.getId() != null && graphInfo.getId() > 0) {
            Long gId = graphInfo.getId();
            return Boolean.TRUE.equals(transactionTemplate.execute(status -> {
                vertexDefService.remove(new QueryWrapper<GraphVertexDef>().eq("graph_id", gId));
                edgeDefService.remove(new QueryWrapper<GraphEdgeDef>().eq("graph_id", gId));
                propertyDefService.remove(new QueryWrapper<GraphPropertyDef>().eq("graph_id", gId));
                return removeById(gId);
            }));
        }

        return true;
    }

    @Override
    public GraphInfo getGraph(Long connectionId, String graphCode) {
        if (connectionId == null && StringUtils.isBlank(graphCode)) {
            return null;
        }
        QueryWrapper<GraphInfo> queryWrapper = new QueryWrapper<>();
        if (connectionId != null) {
            queryWrapper.eq("connection_id", connectionId);
        }

        queryWrapper.eq("code", graphCode);
        return this.getOne(queryWrapper, false);
    }

    private void fillGraphStatus(List<GraphInfo> graphInfos) {
        if (graphInfos == null || graphInfos.isEmpty()) {
            return;
        }
```

```
        Set<Long> connIds = graphInfos.stream().map(GraphInfo::getConnectionId).filter(Objects::nonNull)
                .collect(Collectors.toSet());
        if (CollectionUtils.isEmpty(connIds)) {
            return;
        }

        Map<Long, GraphConnection> connMap = connectionService.listByIds(connIds).stream()
                .collect(Collectors.toMap(GraphConnection::getId, c -> c, (a, b) -> a));

        for (GraphInfo graphInfo : graphInfos) {
            GraphConnection conn = connMap.get(graphInfo.getConnectionId());
            graphInfo.setStatus(conn != null && conn.getStatus() != null && conn.getStatus() == 1 ? 0 : 1);
        }
    }

    private void fillGraphCounts(List<GraphInfo> records) {
        if (records == null || records.isEmpty()) {
            return;
        }
        List<Long> graphIds = records.stream().map(GraphInfo::getId).collect(Collectors.toList());
        List<GraphVertexDef> allVertices = graphVertexDefDao.selectList(
                new QueryWrapper<GraphVertexDef>().in("graph_id", graphIds).select("graph_id"));
        Map<Long, Long> vertexCountMap = allVertices.stream()
                .collect(Collectors.groupingBy(GraphVertexDef::getGraphId, Collectors.counting()));
        List<GraphEdgeDef> allEdges = graphEdgeDefDao.selectList(
                new QueryWrapper<GraphEdgeDef>().in("graph_id", graphIds).select("graph_id"));
        Map<Long, Long> edgeCountMap = allEdges.stream()
                .collect(Collectors.groupingBy(GraphEdgeDef::getGraphId, Collectors.counting()));
        for (GraphInfo graphInfo : records) {
            graphInfo.setVertexTypeCount(vertexCountMap.getOrDefault(graphInfo.getId(), 0L).intValue());
            graphInfo.setEdgeTypeCount(edgeCountMap.getOrDefault(graphInfo.getId(), 0L).intValue());
        }
    }

    /**
     * 从图数据库获取所有图的实时节点/边类型计数
     */
    private void fillCountsFromRemote(Long connectionId, List<GraphInfo> graphInfos) {
        GraphConnection connection = connectionService.getById(connectionId);
        if (connection == null || graphInfos == null || graphInfos.isEmpty()) {
            return;
        }

        if (GraphTypeEnum.janus.name().equalsIgnoreCase(connection.getGraphType())) {
            log.debug("JanusGraph does not support fetching schema stats per graph, skipping");
            return;
        }

        GraphConf graphConf = GraphClientFactory.createGraphConf(connection, "");

        GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
        GraphOperations graphOperations = graphClient.opsForGraph();
        for (GraphInfo g : graphInfos) {
            if (g.getCode() == null) {
                continue;
            }
            try {
                GraphConf schemaConf = GraphClientFactory.createGraphConf(connection, g.getCode());
                com.chenpp.graph.core.schema.GraphSchema schema =
graphOperations.getPublishedSchema(schemaConf);
```

```
                    if (schema != null) {
                        g.setVertexTypeCount(schema.getEntities() != null ? schema.getEntities().size() : 0);
                        g.setEdgeTypeCount(schema.getRelations() != null ? schema.getRelations().size() : 0);
                    }
                } catch (Exception e) {
                    log.debug("获取图类型数量失败，code={}: {}", g.getCode(), e.getMessage());
                }
            }
        }
}

package com.chenpp.graph.admin.model;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableField;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import com.chenpp.graph.admin.enums.GraphTypeEnum;
import lombok.Data;

import java.time.LocalDateTime;

/**
 * 图实体类
 *
 * @author April.Chen
 * @date 2025/8/1 16:31
 */
@TableName("graph")
@Data
public class GraphInfo {

    @TableId(type = IdType.AUTO)
    private Long id;

    private String name;
    private String code;
    private String description;

    /**
     * 状态
     * 0: 正常
     * 1: 异常
     * 2: 未知
     */
    private Integer status;

    /**
     * 关联的图数据库连接 ID
     */
    private Long connectionId;

    /**@
     * 图类型,
     * @see com.chenpp.graph.admin.enums.GraphTypeEnum
     */
    private GraphTypeEnum graphType;
    /**
     * 创建人
     */
```

```
    private String creator;

    private LocalDateTime createTime;
    private LocalDateTime updateTime;

    @TableField(exist = false)
    private Integer vertexCount;

    @TableField(exist = false)
    private Integer edgeCount;

    @TableField(exist = false)
    private Integer vertexTypeCount;

    @TableField(exist = false)
    private Integer edgeTypeCount;

    /** 图来源：PLATFORM-平台创建，EXISTING-图数据库已有 */
    @TableField(exist = false)
    private String sourceType;
}

package com.chenpp.graph.admin.controller;

import com.chenpp.graph.admin.model.*;
import com.chenpp.graph.admin.service.GraphEdgeDefService;
import com.chenpp.graph.admin.service.GraphSchemaService;
import com.chenpp.graph.admin.service.GraphVertexDefService;
import lombok.extern.slf4j.Slf4j;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Objects;
import java.util.Set;
import java.util.stream.Collectors;

/**
 * 图 Schema 管理控制器
 */
@Slf4j
@RestController
@RequestMapping("/api/graphs/schema")
public class GraphSchemaController {

    @Autowired
    private GraphVertexDefService vertexDefService;

    @Autowired
    private GraphEdgeDefService edgeDefService;
```

```
    @Autowired
    private GraphSchemaService graphSchemaService;

    @GetMapping("/vertices")
    public Result<List<GraphVertexDef>> getVertexDefs(
            @RequestParam Long graphId, Integer status,
            @RequestParam(required = false) Long connectionId,
            @RequestParam(required = false) String graphCode) {
        List<GraphVertexDef> vertexDefs = vertexDefService.getVertexDefsByGraphId(graphId, status);
        if (vertexDefs == null) {
            vertexDefs = new ArrayList<>();
        }

        List<GraphVertexDef> discovered = graphSchemaService.discoverVertexDefs(graphId, connectionId, graphCode);
        mergeDiscoveredDefs(vertexDefs, discovered);

        return Result.success(vertexDefs);
    }

    @PostMapping("/vertices")
    public Result<String> addVertexDef(@RequestBody GraphVertexDef vertexDef) {
        boolean success = vertexDefService.saveVertexDefWithProperties(vertexDef);
        if (success) {
            return Result.success("新增节点定义成功");
        }
        return Result.error("新增节点定义失败");
    }

    @PutMapping("/vertex")
    public Result<String> updateVertexDef(
            @RequestParam Long vertexId,
            @RequestBody GraphVertexDef vertexDef) {
        boolean success;
        if (vertexId != null && vertexId > 0) {
            vertexDef.setId(vertexId);
            success = vertexDefService.updateVertexDefWithProperties(vertexDef);
        } else {
            vertexDef.setId(null);
            success = vertexDefService.saveVertexDefWithProperties(vertexDef);
        }

        if (success) {
            return Result.success(vertexId != null && vertexId > 0 ? "更新节点定义成功" : "新增节点定义成功");
        }
        return Result.error(vertexId != null && vertexId > 0 ? "更新节点定义失败" : "新增节点定义失败");
    }

    @DeleteMapping("/vertex")
    public Result<String> deleteVertexDef(@RequestParam Long graphId, @RequestParam Long vertexId) {
        boolean success = graphSchemaService.deleteVertexDef(graphId, vertexId);
        if (success) {
            return Result.success("删除节点定义成功");
        }
        return Result.error("删除节点定义失败");
    }

    @GetMapping("/edges")
    public Result<List<GraphEdgeDef>> getEdgeDefs(
            @RequestParam Long graphId, Integer status,
            @RequestParam(required = false) Long connectionId,
```

```
            @RequestParam(required = false) String graphCode) {
        List<GraphEdgeDef> edgeDefs = edgeDefService.getEdgeDefsByGraphId(graphId, status);
        if (edgeDefs == null) {
            edgeDefs = new ArrayList<>();
        }

        List<GraphEdgeDef> discovered = graphSchemaService.discoverEdgeDefs(graphId, connectionId, graphCode);
        mergeDiscoveredDefs(edgeDefs, discovered);

        return Result.success(edgeDefs);
    }

    @PostMapping("/edges")
    public Result<String> addEdgeDef(@RequestBody GraphEdgeDef edgeDef) {
        boolean success = edgeDefService.saveEdgeDefWithProperties(edgeDef);
        if (success) {
            return Result.success("新增边定义成功");
        }
        return Result.error("新增边定义失败");
    }

    @PutMapping("/edge")
    public Result<String> updateEdgeDef(@RequestParam Long edgeId, @RequestBody GraphEdgeDef edgeDef) {
        edgeDef.setId(edgeId);
        boolean success = edgeDefService.updateEdgeDefWithProperties(edgeDef);
        if (success) {
            return Result.success("更新边定义成功");
        }
        return Result.error("更新边定义失败");
    }

    @DeleteMapping("/edge")
    public Result<String> deleteEdgeDef(@RequestParam Long graphId, @RequestParam Long edgeId) {
        boolean success = graphSchemaService.deleteEdgeDef(graphId, edgeId);
        if (success) {
            return Result.success("删除边定义成功");
        }
        return Result.error("删除边定义失败");
    }

    @PostMapping("/publish")
    public Result<String> publishSchema(@RequestParam Long graphId) {
        try {
            graphSchemaService.publishSchema(graphId, null, null);
            return Result.success("发布成功");
        } catch (Exception e) {
            log.error("发布图 Schema 失败，graphId={}", graphId, e);
            return Result.error(500, "发布 Schema 失败: " + e.getMessage(), null);
        }
    }

    @GetMapping("/export")
    public Result<SchemaExportResponse> exportSchema(@RequestParam Long graphId) {
        return Result.success(graphSchemaService.exportSchema(graphId));
    }

    @PostMapping("/import")
    public Result<String> importSchema(@RequestBody SchemaImportRequest importRequest) {
        graphSchemaService.importSchema(importRequest);
        return Result.success("导入成功");
```

```
    }

    private <T extends GraphEntityDef> void mergeDiscoveredDefs(List<T> localDefs, List<T> discovered) {
        if (discovered == null || discovered.isEmpty()) {
            return;
        }

        Set<String> localLabels = localDefs.stream()
                .map(GraphEntityDef::getLabel)
                .filter(Objects::nonNull)
                .collect(Collectors.toSet());

        for (T d : discovered) {
            if (d.getLabel() != null && !localLabels.contains(d.getLabel())) {
                localDefs.add(d);
                localLabels.add(d.getLabel());
            }
        }

        Map<String, List<GraphPropertyDef>> discoveredProps = discovered.stream()
                .filter(d -> d.getLabel() != null && d.getProperties() != null)
                .collect(Collectors.toMap(GraphEntityDef::getLabel, GraphEntityDef::getProperties, (a, b) -> a));
        for (T def : localDefs) {
            if (def.getLabel() == null || !discoveredProps.containsKey(def.getLabel())) {
                continue;
            }
            List<GraphPropertyDef> existing = def.getProperties() != null
                    ? new ArrayList<>(def.getProperties()) : new ArrayList<>();
            Set<String> existingCodes = existing.stream()
                    .map(GraphPropertyDef::getCode)
                    .filter(Objects::nonNull)
                    .collect(Collectors.toSet());
            for (GraphPropertyDef p : discoveredProps.get(def.getLabel())) {
                if (p.getCode() != null && !existingCodes.contains(p.getCode())) {
                    existing.add(p);
                }
            }
            def.setProperties(existing);
        }
    }
}

package com.chenpp.graph.admin.service.impl;

import com.baomidou.mybatisplus.core.conditions.query.QueryWrapper;
import com.chenpp.graph.admin.model.GraphConnection;
import com.chenpp.graph.admin.model.GraphEdgeDef;
import com.chenpp.graph.admin.model.GraphInfo;
import com.chenpp.graph.admin.model.GraphPropertyDef;
import com.chenpp.graph.admin.model.GraphVertexDef;
import com.chenpp.graph.admin.model.SchemaExportResponse;
import com.chenpp.graph.admin.model.SchemaImportRequest;
import com.chenpp.graph.admin.service.GraphConnectionService;
import com.chenpp.graph.admin.service.GraphEdgeDefService;
import com.chenpp.graph.admin.service.GraphPropertyDefService;
import com.chenpp.graph.admin.service.GraphSchemaService;
import com.chenpp.graph.admin.service.GraphService;
import com.chenpp.graph.admin.service.GraphVertexDefService;
import com.chenpp.graph.admin.util.GraphClientFactory;
import com.chenpp.graph.core.GraphClient;
```

```
import com.chenpp.graph.core.GraphOperations;
import com.chenpp.graph.core.constant.GraphConstants;
import com.chenpp.graph.core.exception.GraphException;
import com.chenpp.graph.core.model.GraphConf;
import com.chenpp.graph.core.schema.DataType;
import com.chenpp.graph.core.schema.GraphEntity;
import com.chenpp.graph.core.schema.GraphIndex;
import com.chenpp.graph.core.schema.GraphProperty;
import com.chenpp.graph.core.schema.GraphRelation;
import com.chenpp.graph.core.schema.GraphSchema;
import com.chenpp.graph.core.schema.IndexType;
import lombok.extern.slf4j.Slf4j;
import org.apache.commons.collections4.CollectionUtils;
import org.apache.commons.lang3.StringUtils;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.transaction.support.TransactionTemplate;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.atomic.AtomicLong;
import java.util.stream.Collectors;


@Slf4j
@Service
public class GraphSchemaServiceImpl implements GraphSchemaService {


    @Autowired
    private GraphService graphService;

    @Autowired
    private GraphConnectionService connectionService;

    @Autowired
    private GraphVertexDefService graphVertexDefService;

    @Autowired
    private GraphEdgeDefService graphEdgeDefService;

    @Autowired
    private GraphPropertyDefService graphPropertyDefService;

    @Autowired
    private TransactionTemplate transactionTemplate;


    @Override
    public GraphSchema discoverSchema(Long connectionId, String graphCode, Long graphId) {
        if (graphId != null && graphId > 0) {
            GraphInfo graphInfo = graphService.getById(graphId);
            if (graphInfo == null) {
                throw new GraphException("图不存在");
            }
            graphCode = graphInfo.getCode();
```

```
            connectionId = graphInfo.getConnectionId();
        }
        GraphConnection connection = connectionService.getById(connectionId);
        if (connection == null) {
            throw new GraphException("图数据库连接不存在");
        }

        GraphConf graphConf = GraphClientFactory.createGraphConf(connection, graphCode);
        GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
        GraphOperations graphOperations = graphClient.opsForGraph();
        return graphOperations.getPublishedSchema(graphConf);
    }


    @Override
    public List<GraphVertexDef> discoverVertexDefs(Long graphId, Long connectionId, String graphCode) {
        GraphSchema schema = discoverSchema(connectionId, graphCode, graphId);
        if (schema == null || CollectionUtils.isEmpty(schema.getEntities())) {
            return new ArrayList<>();
        }
        AtomicLong idCounter = new AtomicLong(-1);
        return schema.getEntities().stream()
                .map(entity -> buildVertexDef(entity, graphId, idCounter))
                .collect(Collectors.toList());
    }

    @Override
    public List<GraphEdgeDef> discoverEdgeDefs(Long graphId, Long connectionId, String graphCode) {
        GraphSchema schema = discoverSchema(connectionId, graphCode, graphId);
        if (schema == null || CollectionUtils.isEmpty(schema.getRelations())) {
            return new ArrayList<>();
        }
        AtomicLong idCounter = new AtomicLong(-1000);
        return schema.getRelations().stream()
                .map(relation -> buildEdgeDef(relation, graphId, idCounter))
                .collect(Collectors.toList());
    }

    private GraphVertexDef buildVertexDef(GraphEntity entity, Long graphId, AtomicLong idCounter) {
        GraphVertexDef vertexDef = new GraphVertexDef();
        vertexDef.setId(idCounter.decrementAndGet());
        vertexDef.setGraphId(graphId);
        vertexDef.setLabel(entity.getLabel());
        vertexDef.setName(entity.getLabel());
        vertexDef.setStatus(1);
        vertexDef.setProperties(buildPropertyDefs(entity.getProperties()));
        return vertexDef;
    }

    private GraphEdgeDef buildEdgeDef(GraphRelation relation, Long graphId, AtomicLong idCounter) {
        GraphEdgeDef edgeDef = new GraphEdgeDef();
        edgeDef.setId(idCounter.decrementAndGet());
        edgeDef.setGraphId(graphId);
        edgeDef.setLabel(relation.getLabel());
        edgeDef.setName(relation.getLabel());
        edgeDef.setStatus(1);
        edgeDef.setMultiple(relation.getMultiple());
        edgeDef.setStartLabel(relation.getStartLabel());
        edgeDef.setEndLabel(relation.getEndLabel());
        edgeDef.setProperties(buildPropertyDefs(relation.getProperties()));
```

```
        return edgeDef;
    }

    private List<GraphPropertyDef> buildPropertyDefs(List<GraphProperty> properties) {
        if (CollectionUtils.isEmpty(properties)) {
            return new ArrayList<>();
        }
        return properties.stream()
                .map(this::buildPropertyDef)
                .collect(Collectors.toList());
    }

    private GraphPropertyDef buildPropertyDef(GraphProperty p) {
        GraphPropertyDef prop = new GraphPropertyDef();
        prop.setCode(p.getCode());
        prop.setName(p.getName());
        prop.setType(p.getDataType() != null ? p.getDataType().name() : "String");
        prop.setStatus(1);
        prop.setIndexed(false);
        return prop;
    }

    @Override
    public GraphSchema getGraphSchema(Long graphId) {
        GraphInfo graphInfo = graphService.getById(graphId);
        if (graphInfo == null) {
            log.warn("图不存在，返回空 Schema，graphId={}", graphId);
            return new GraphSchema();
        }
        List<GraphVertexDef> vertices = graphVertexDefService.getVertexDefsByGraphId(graphId, null);
        List<GraphEdgeDef> edges = graphEdgeDefService.getEdgeDefsByGraphId(graphId, null);
        GraphSchema graphSchema = new GraphSchema();
        graphSchema.setGraphCode(graphInfo.getCode());

        List<GraphIndex> indexes = new ArrayList<>();

        List<GraphEntity> entities = vertices.stream().map(vertex -> {
            GraphEntity entity = new GraphEntity();
            entity.setLabel(vertex.getLabel());
            List<GraphProperty> properties = transformGraphProperty(vertex.getProperties());
            properties.forEach(p -> {
                if (p.getIndexed()) {
                    GraphIndex index = transformGraphIndex(graphInfo.getCode(), entity.getLabel(),
GraphConstants.VERTEX, p);
                    indexes.add(index);
                }
            });

            entity.setProperties(properties);
            return entity;
        }).toList();

        List<GraphRelation> relations = edges.stream().map(edge -> {
            GraphRelation relation = new GraphRelation();
            relation.setLabel(edge.getLabel());
            relation.setStartLabel(edge.getStartLabel());
            relation.setEndLabel(edge.getEndLabel());
            relation.setMultiple(edge.getMultiple());
            List<GraphProperty> properties = transformGraphProperty(edge.getProperties());
            properties.forEach(p -> {
```

```
                if (p.getIndexed()) {
                    GraphIndex index = transformGraphIndex(graphInfo.getCode(), edge.getLabel(),
GraphConstants.EDGE, p);
                    indexes.add(index);
                }
            });

            relation.setProperties(properties);

            return relation;
        }).toList();
        graphSchema.setEntities(entities);
        graphSchema.setRelations(relations);
        graphSchema.setIndexes(indexes);
        return graphSchema;
    }

    @Override
    public void publishSchema(Long graphId, Long connectionId, String graphCode) {
        log.info("发布图 Schema: graphId={}, connectionId={}, graphCode={}", graphId, connectionId, graphCode);

        GraphConf graphConf;
        GraphInfo graphInfo;
        GraphConnection connection;

        if (graphId != null && graphId > 0) {
            graphInfo = graphService.getById(graphId);
            if (graphInfo == null) {
                log.warn("图不存在，跳过发布 Schema，graphId={}", graphId);
                return;
            }
            connection = connectionService.getById(graphInfo.getConnectionId());
            if (connection == null) {
                log.error("图数据库连接不存在，connectionId={}", graphInfo.getConnectionId());
                return;
            }
            graphConf = GraphClientFactory.createGraphConf(connection, graphInfo.getCode());
        } else {
            graphInfo = null;
            if (connectionId == null || graphCode == null) {
                log.warn("Discovered graph 需要 connectionId 和 graphCode，跳过发布 Schema");
                return;
            }
            graphConf = GraphClientFactory.resolveGraphConf(graphId, connectionId, graphCode, graphService,
connectionService);
        }

        List<GraphVertexDef> vertices = graphVertexDefService.getVertexDefsByGraphId(graphId, null);
        List<GraphEdgeDef> edges = graphEdgeDefService.getEdgeDefsByGraphId(graphId, null);

        GraphSchema graphSchema = getGraphSchema(graphId);

        GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
        GraphOperations graphOperations = graphClient.opsForGraph();

        GraphSchema remoteSchema;
        try {
            remoteSchema = graphOperations.getPublishedSchema(graphConf);
        } catch (Exception e) {
            log.warn("获取远程 Schema 失败，将全量发布: {}", e.getMessage());
```

```
        remoteSchema = new GraphSchema();
    }

    Set<String> remoteLabelNames = remoteSchema.getEntities() != null
            ? remoteSchema.getEntities().stream().map(GraphEntity::getLabel).collect(Collectors.toSet())
            : Set.of();
    Set<String> remoteEdgeNames = remoteSchema.getRelations() != null
            ? remoteSchema.getRelations().stream().map(GraphRelation::getLabel).collect(Collectors.toSet())
            : Set.of();
    Set<String> remoteIndexNames = remoteSchema.getIndexes() != null
            ? remoteSchema.getIndexes().stream().map(GraphIndex::getName).collect(Collectors.toSet())
            : Set.of();

    List<GraphEntity> newEntities = new ArrayList<>();
    List<GraphEntity> alterEntities = new ArrayList<>();
    if (graphSchema.getEntities() != null) {
        for (GraphEntity entity : graphSchema.getEntities()) {
            if (remoteLabelNames.contains(entity.getLabel())) {
                alterEntities.add(entity);
            } else {
                newEntities.add(entity);
            }
        }
    }

    List<GraphRelation> newRelations = new ArrayList<>();
    List<GraphRelation> alterRelations = new ArrayList<>();
    if (graphSchema.getRelations() != null) {
        for (GraphRelation relation : graphSchema.getRelations()) {
            if (remoteEdgeNames.contains(relation.getLabel())) {
                alterRelations.add(relation);
            } else {
                newRelations.add(relation);
            }
        }
    }

    List<GraphIndex> newIndexes = new ArrayList<>();
    if (graphSchema.getIndexes() != null) {
        for (GraphIndex idx : graphSchema.getIndexes()) {
            if (!remoteIndexNames.contains(idx.getName())) {
                newIndexes.add(idx);
            }
        }
    }

    Set<String> newEntityLabels = newEntities.stream().map(GraphEntity::getLabel).collect(Collectors.toSet());
    Set<String> newRelationLabels = newRelations.stream().map(GraphRelation::getLabel).collect(Collectors.toSet());
    List<GraphIndex> newEntityIndexes = new ArrayList<>();
    List<GraphIndex> alterEntityIndexes = new ArrayList<>();
    for (GraphIndex idx : newIndexes) {
        boolean isNewEntity = GraphConstants.VERTEX.equalsIgnoreCase(idx.getSchemaType())
                ? newEntityLabels.contains(idx.getLabel())
                : newRelationLabels.contains(idx.getLabel());
        if (isNewEntity) {
            newEntityIndexes.add(idx);
        } else {
            alterEntityIndexes.add(idx);
        }
    }
```

```
        GraphSchema newSchema = new GraphSchema();
        newSchema.setEntities(newEntities);
        newSchema.setRelations(newRelations);
        newSchema.setIndexes(newEntityIndexes);

        GraphSchema alterSchema = new GraphSchema();
        alterSchema.setEntities(alterEntities);
        alterSchema.setRelations(alterRelations);

        boolean hasNew = !newEntities.isEmpty() || !newRelations.isEmpty() || !newEntityIndexes.isEmpty();
        boolean hasAlter = !alterEntities.isEmpty() || !alterRelations.isEmpty() || !alterEntityIndexes.isEmpty();

        if (hasNew) {
            graphOperations.applySchema(graphConf, newSchema);
        }
        if (hasAlter) {
            graphOperations.alterSchema(graphConf, alterSchema);
        }
        if (!alterEntityIndexes.isEmpty()) {
            GraphSchema alterIndexSchema = new GraphSchema();
            alterIndexSchema.setIndexes(alterEntityIndexes);
            try {
                graphOperations.applySchema(graphConf, alterIndexSchema);
                Thread.sleep(30 * 1000);
            } catch (InterruptedException ie) {
                Thread.currentThread().interrupt();
            }
        }

        if (graphInfo != null) {
            List<GraphPropertyDef> propertyList = new ArrayList<>();
            transactionTemplate.executeWithoutResult(status -> {
                vertices.forEach(vertex -> {
                    vertex.setStatus(1);
                    propertyList.addAll(vertex.getProperties());
                });
                graphVertexDefService.updateBatchById(vertices);

                edges.forEach(edge -> {
                    edge.setStatus(1);
                    propertyList.addAll(edge.getProperties());
                });
                graphEdgeDefService.updateBatchById(edges);
                propertyList.forEach(p -> p.setStatus(1));
                graphPropertyDefService.updateBatchById(propertyList);

                graphInfo.setStatus(1);
                graphService.updateById(graphInfo);
            });
        }
    }

    @Override
    public void publishVertexDef(Long graphId, Long connectionId, String graphCode, String label) {
        if ((connectionId == null || graphCode == null) && graphId != null) {
            GraphInfo graphInfo = graphService.getById(graphId);
            if (graphInfo != null) {
                connectionId = graphInfo.getConnectionId();
```

```
                graphCode = graphInfo.getCode();
            }
        }
        GraphVertexDef vertexDef = null;
        if (graphId != null && label != null) {
            vertexDef = graphVertexDefService.getOne(
                    new QueryWrapper<GraphVertexDef>().eq("graph_id", graphId).eq("label", label));
        }

        GraphEntity entity = new GraphEntity();
        entity.setLabel(label);
        if (vertexDef != null && vertexDef.getProperties() != null) {
            entity.setProperties(transformGraphProperty(vertexDef.getProperties()));
        }

        GraphSchema schema = new GraphSchema();
        schema.setEntities(List.of(entity));

        publishSingleSchema(connectionId, graphCode, schema);

        if (vertexDef != null) {
            vertexDef.setStatus(1);
            graphVertexDefService.updateById(vertexDef);
        }
    }

    @Override
    public void publishEdgeDef(Long graphId, Long connectionId, String graphCode, String label) {
        if ((connectionId == null || graphCode == null) && graphId != null) {
            GraphInfo graphInfo = graphService.getById(graphId);
            if (graphInfo != null) {
                connectionId = graphInfo.getConnectionId();
                graphCode = graphInfo.getCode();
            }
        }
        GraphEdgeDef edgeDef = null;
        if (graphId != null && label != null) {
            edgeDef = graphEdgeDefService.getOne(
                    new QueryWrapper<GraphEdgeDef>().eq("graph_id", graphId).eq("label", label));
        }

        GraphRelation relation = new GraphRelation();
        relation.setLabel(label);
        if (edgeDef != null) {
            relation.setStartLabel(edgeDef.getStartLabel());
            relation.setEndLabel(edgeDef.getEndLabel());
            relation.setMultiple(edgeDef.getMultiple());
            if (edgeDef.getProperties() != null) {
                relation.setProperties(transformGraphProperty(edgeDef.getProperties()));
            }
        }

        GraphSchema schema = new GraphSchema();
        schema.setRelations(List.of(relation));

        publishSingleSchema(connectionId, graphCode, schema);

        if (edgeDef != null) {
            edgeDef.setStatus(1);
            graphEdgeDefService.updateById(edgeDef);
```

```
        }
    }

    private void publishSingleSchema(Long connectionId, String graphCode, GraphSchema schema) {
        GraphConnection connection = connectionService.getById(connectionId);
        if (connection == null) {
            log.error("图数据库连接不存在，connectionId={}", connectionId);
            return;
        }
        GraphConf graphConf = GraphClientFactory.createGraphConf(connection, graphCode);

        GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
        GraphOperations graphOperations = graphClient.opsForGraph();

        GraphSchema remoteSchema;
        try {
            remoteSchema = graphOperations.getPublishedSchema(graphConf);
        } catch (Exception e) {
            log.warn("获取远程 Schema 失败，将全量发布: {}", e.getMessage());
            remoteSchema = new GraphSchema();
        }

        boolean hasNew = false;
        boolean hasAlter = false;
        GraphSchema newSchema = new GraphSchema();
        GraphSchema alterSchema = new GraphSchema();

        if (schema.getEntities() != null && !schema.getEntities().isEmpty()) {
            Set<String> remoteLabels = remoteSchema.getEntities() != null
                    ? remoteSchema.getEntities().stream().map(GraphEntity::getLabel).collect(Collectors.toSet())
                    : Set.of();
            List<GraphEntity> newEntities = new ArrayList<>();
            List<GraphEntity> alterEntities = new ArrayList<>();
            for (GraphEntity entity : schema.getEntities()) {
                if (remoteLabels.contains(entity.getLabel())) {
                    alterEntities.add(entity);
                } else {
                    newEntities.add(entity);
                }
            }
            newSchema.setEntities(newEntities);
            alterSchema.setEntities(alterEntities);
            if (!newEntities.isEmpty()) {
                hasNew = true;
            }
            if (!alterEntities.isEmpty()) {
                hasAlter = true;
            }
        }

        if (schema.getRelations() != null && !schema.getRelations().isEmpty()) {
            Set<String> remoteEdges = remoteSchema.getRelations() != null
                    ? remoteSchema.getRelations().stream().map(GraphRelation::getLabel).collect(Collectors.toSet())
                    : Set.of();
            List<GraphRelation> newRelations = new ArrayList<>();
            List<GraphRelation> alterRelations = new ArrayList<>();
            for (GraphRelation relation : schema.getRelations()) {
                if (remoteEdges.contains(relation.getLabel())) {
                    alterRelations.add(relation);
                } else {
```

```
                    newRelations.add(relation);
                }
            }
            newSchema.setRelations(newRelations);
            alterSchema.setRelations(alterRelations);
            if (!newRelations.isEmpty()) {
                hasNew = true;
            }
            if (!alterRelations.isEmpty()) {
                hasAlter = true;
            }
        }

        if (hasNew) {
            graphOperations.applySchema(graphConf, newSchema);
        }
        if (hasAlter) {
            graphOperations.alterSchema(graphConf, alterSchema);
        }

        log.info("增量 Schema 发布完成");
    }

    public List<GraphProperty> transformGraphProperty(List<GraphPropertyDef> properties) {
        return properties.stream().map(prop -> {
            GraphProperty property = new GraphProperty();
            property.setName(prop.getName());
            property.setCode(prop.getCode());
            String typeStr = prop.getType();
            DataType dataType = DataType.instanceOf(typeStr);
            if (dataType == null) {
                if (StringUtils.isNotBlank(typeStr)) {
                    log.warn("Failed to parse data type '{}' for property '{}', using String as default", typeStr,
prop.getCode());
                }
                dataType = DataType.String; // 设置默认值，避免 NPE
            }
            property.setDataType(dataType);
            property.setIndexed(prop.getIndexed());
            return property;
        }).collect(Collectors.toList());
    }

    @Override
    public SchemaExportResponse exportSchema(Long graphId) {
        GraphInfo graphInfo = graphService.getById(graphId);
        if (graphInfo == null) {
            throw new GraphException("图不存在");
        }
        List<GraphVertexDef> vertices = graphVertexDefService.getVertexDefsByGraphId(graphId, null);
        List<GraphEdgeDef> edges = graphEdgeDefService.getEdgeDefsByGraphId(graphId, null);

        SchemaExportResponse dto = new SchemaExportResponse();
        dto.setExportedAt(LocalDateTime.now().toString());
        dto.setGraphId(graphId);
        dto.setGraphCode(graphInfo.getCode());
        dto.setVertices(vertices);
        dto.setEdges(edges);
        return dto;
    }
```

```
@Transactional(rollbackFor = Exception.class)
@Override
public void importSchema(SchemaImportRequest importRequest) {
    Long graphId = importRequest.getGraphId();
    if (importRequest == null) {
        throw new GraphException("导入数据为空");
    }

    List<GraphVertexDef> importVertices = importRequest.getVertices();
    List<GraphEdgeDef> importEdges = importRequest.getEdges();
    if (CollectionUtils.isEmpty(importVertices) && CollectionUtils.isEmpty(importEdges)) {
        throw new GraphException("导入数据为空");
    }

    String mode = importRequest.getMode();
    boolean replace = "replace".equalsIgnoreCase(mode);

    if (replace) {
        List<GraphVertexDef> existingVertices = graphVertexDefService.getVertexDefsByGraphId(graphId, null);
        for (GraphVertexDef vertex : existingVertices) {
            graphVertexDefService.deleteVertexDefWithProperties(vertex.getId());
        }
        List<GraphEdgeDef> existingEdges = graphEdgeDefService.getEdgeDefsByGraphId(graphId, null);
        for (GraphEdgeDef edge : existingEdges) {
            graphEdgeDefService.deleteEdgeDefWithProperties(edge.getId());
        }
    }

    Map<String, Long> existingVertexLabelIdMap;
    if (replace) {
        existingVertexLabelIdMap = Map.of();
    } else {
        List<GraphVertexDef> existingVertices = graphVertexDefService.getVertexDefsByGraphId(graphId, null);
        existingVertexLabelIdMap = existingVertices.stream()
                .filter(n -> n.getLabel() != null)
                .collect(Collectors.toMap(GraphVertexDef::getLabel, GraphVertexDef::getId, (a, b) -> a));
    }

    if (importVertices != null) {
        for (GraphVertexDef vertex : importVertices) {
            if (!replace && existingVertexLabelIdMap.containsKey(vertex.getLabel())) {
                continue;
            }
            vertex.setGraphId(graphId);
            vertex.setId(null);
            vertex.setStatus(0);
            graphVertexDefService.saveVertexDefWithProperties(vertex);
        }
    }


    if (importEdges != null) {
        Map<String, Boolean> existingEdgeLabelMap;
        if (replace) {
            existingEdgeLabelMap = Map.of();
        } else {
            existingEdgeLabelMap = graphEdgeDefService.getEdgeDefsByGraphId(graphId, null).stream()
                    .filter(e -> e.getLabel() != null)
                    .collect(Collectors.toMap(GraphEdgeDef::getLabel, e -> true, (a, b) -> a));
```

```
            }
            for (GraphEdgeDef edge : importEdges) {
                if (!replace && existingEdgeLabelMap.containsKey(edge.getLabel())) {
                    continue;
                }
                edge.setGraphId(graphId);
                edge.setId(null);
                edge.setStatus(0);
                graphEdgeDefService.saveEdgeDefWithProperties(edge);
            }
        }

        log.info("Schema 导入完成，graphId={}, mode={}, vertices={}, edges={}", graphId, mode,
                importVertices != null ? importVertices.size() : 0,
                importEdges != null ? importEdges.size() : 0);
    }

    @Override
    public Long createGraphInDatabase(GraphInfo graphInfo) {
        if (graphInfo == null) {
            throw new GraphException("图信息为空");
        }
        GraphConnection connection = connectionService.getById(graphInfo.getConnectionId());
        if (connection == null) {
            throw new GraphException("图数据库连接不存在");
        }

        GraphConf graphConf = GraphClientFactory.createGraphConf(connection, graphInfo.getCode());
        GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
        GraphOperations graphOperations = graphClient.opsForGraph();

        List<com.chenpp.graph.core.schema.Graph> existingGraphs = graphOperations.listGraphs(graphConf);
        boolean graphExists = existingGraphs.stream()
                .anyMatch(g -> graphInfo.getCode().equals(g.getCode()));
        if (!graphExists) {
            graphOperations.createGraph(graphConf);
            log.info("已在图数据库中创建图，graphCode={}", graphInfo.getCode());
        } else {
            log.info("图已存在于图数据库中，跳过创建，graphCode={}", graphInfo.getCode());
        }

        graphInfo.setStatus(0);
        graphInfo.setGraphType(connection.getGraphTypeEnum());
        graphService.save(graphInfo);
        log.info("已保存图到 MySQL，graphId={}", graphInfo.getId());

        return graphInfo.getId();
    }

    @Transactional(rollbackFor = Exception.class)
    @Override
    public boolean deleteVertexDef(Long graphId, Long vertexId) {
        log.info("删除节点定义: graphId={}, vertexId={}", graphId, vertexId);

        GraphVertexDef vertexDef = graphVertexDefService.getById(vertexId);
        if (vertexDef == null) {
            log.warn("顶点定义不存在，vertexId={}", vertexId);
            return false;
        }
```

```
        GraphInfo graphInfo = graphService.getById(graphId);
        if (graphInfo != null && graphInfo.getConnectionId() != null) {
            GraphConnection connection = connectionService.getById(graphInfo.getConnectionId());
            if (connection != null) {
                GraphConf graphConf = GraphClientFactory.createGraphConf(connection, graphInfo.getCode());
                GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
                GraphOperations graphOperations = graphClient.opsForGraph();

                graphOperations.dropVertexLabel(graphInfo.getCode(), vertexDef.getLabel());
                log.info("已在图数据库中删除顶点标签: {}", vertexDef.getLabel());
            }
        }

        return graphVertexDefService.deleteVertexDefWithProperties(vertexId);
    }

    @Transactional(rollbackFor = Exception.class)
    @Override
    public boolean deleteEdgeDef(Long graphId, Long edgeId) {
        log.info("删除边定义: graphId={}, edgeId={}", graphId, edgeId);

        GraphEdgeDef edgeDef = graphEdgeDefService.getById(edgeId);
        if (edgeDef == null) {
            log.warn("边定义不存在，edgeId={}", edgeId);
            return false;
        }

        GraphInfo graphInfo = graphService.getById(graphId);
        if (graphInfo != null && graphInfo.getConnectionId() != null) {
            GraphConnection connection = connectionService.getById(graphInfo.getConnectionId());
            if (connection != null) {
                GraphConf graphConf = GraphClientFactory.createGraphConf(connection, graphInfo.getCode());
                GraphClient graphClient = GraphClientFactory.createGraphClient(graphConf);
                GraphOperations graphOperations = graphClient.opsForGraph();

                graphOperations.dropEdgeLabel(graphInfo.getCode(), edgeDef.getLabel());
                log.info("已在图数据库中删除边类型: {}", edgeDef.getLabel());
            }
        }

        return graphEdgeDefService.deleteEdgeDefWithProperties(edgeId);
    }

    private GraphIndex transformGraphIndex(String graphCode, String label, String schemaType, GraphProperty property)
{
        GraphIndex index = new GraphIndex();
        index.setLabel(label);
        index.setProperty(property.getCode());
        index.setType(IndexType.COMPOSITE.code());
        index.setSchemaType(schemaType);
        index.setPropertyNames(List.of(property.getCode()));
        index.setName(String.format("idx_%s_%s_%s", graphCode, label, property.getCode()));
        index.setProperties(List.of(property));
        return index;
    }
}

package com.chenpp.graph.admin.service.impl;

import com.chenpp.graph.admin.mapper.GraphVertexDefDao;
```

```
import com.chenpp.graph.admin.model.GraphVertexDef;
import com.chenpp.graph.admin.service.GraphSchemaService;
import com.chenpp.graph.admin.service.GraphVertexDefService;
import com.chenpp.graph.core.constant.GraphConstants;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Lazy;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

/**
 * 图节点定义服务实现类
 */
@Service
public class GraphVertexDefServiceImpl extends AbstractEntityDefService<GraphVertexDefDao, GraphVertexDef>
        implements GraphVertexDefService {

    @Lazy
    @Autowired
    private GraphSchemaService graphSchemaService;

    @Override
    protected String getPropertyType() {
        return GraphConstants.VERTEX;
    }

    @Override
    public List<GraphVertexDef> getVertexDefsByGraphId(Long graphId, Integer status) {
        return getEntityDefsByGraphId(graphId, status);
    }

    @Override
    @Transactional(rollbackFor = Exception.class)
    public boolean saveVertexDefWithProperties(GraphVertexDef vertexDef) {
        boolean saved = saveEntityDefWithProperties(vertexDef);
        if (saved && vertexDef.getGraphId() != null) {
            graphSchemaService.publishVertexDef(vertexDef.getGraphId(), null, null, vertexDef.getLabel());
        }
        return saved;
    }

    @Override
    @Transactional(rollbackFor = Exception.class)
    public boolean updateVertexDefWithProperties(GraphVertexDef vertexDef) {
        boolean updated = updateEntityDefWithProperties(vertexDef);
        if (updated && vertexDef.getGraphId() != null) {
            graphSchemaService.publishVertexDef(vertexDef.getGraphId(), null, null, vertexDef.getLabel());
        }
        return updated;
    }

    @Override
    @Transactional(rollbackFor = Exception.class)
    public boolean deleteVertexDefWithProperties(Long id) {
        return deleteEntityDefWithProperties(id);
    }
}

package com.chenpp.graph.admin.service.impl;
```

```
import com.chenpp.graph.admin.mapper.GraphEdgeDefDao;
import com.chenpp.graph.admin.model.GraphEdgeDef;
import com.chenpp.graph.admin.service.GraphEdgeDefService;
import com.chenpp.graph.admin.service.GraphSchemaService;
import com.chenpp.graph.core.constant.GraphConstants;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.context.annotation.Lazy;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

/**
 * 图边定义服务实现类
 *
 * @author April.Chen
 * @date 2025/8/4 16:00
 */
@Service
public class GraphEdgeDefServiceImpl extends AbstractEntityDefService<GraphEdgeDefDao, GraphEdgeDef>
        implements GraphEdgeDefService {

    @Lazy
    @Autowired
    private GraphSchemaService graphSchemaService;

    @Override
    protected String getPropertyType() {
        return GraphConstants.EDGE;
    }

    @Override
    public List<GraphEdgeDef> getEdgeDefsByGraphId(Long graphId, Integer status) {
        return getEntityDefsByGraphId(graphId, status);
    }

    @Override
    @Transactional(rollbackFor = Exception.class)
    public boolean saveEdgeDefWithProperties(GraphEdgeDef edgeDef) {
        boolean saved = saveEntityDefWithProperties(edgeDef);
        if (saved && edgeDef.getGraphId() != null) {
            graphSchemaService.publishEdgeDef(edgeDef.getGraphId(), null, null, edgeDef.getLabel());
        }
        return saved;
    }

    @Override
    @Transactional(rollbackFor = Exception.class)
    public boolean updateEdgeDefWithProperties(GraphEdgeDef edgeDef) {
        boolean updated = updateEntityDefWithProperties(edgeDef);
        if (updated && edgeDef.getGraphId() != null) {
            graphSchemaService.publishEdgeDef(edgeDef.getGraphId(), null, null, edgeDef.getLabel());
        }
        return updated;
    }

    @Override
    @Transactional(rollbackFor = Exception.class)
    public boolean deleteEdgeDefWithProperties(Long id) {
```

```
            return deleteEntityDefWithProperties(id);
        }
}
package com.chenpp.graph.admin.model;

import com.baomidou.mybatisplus.annotation.IdType;
import com.baomidou.mybatisplus.annotation.TableField;
import com.baomidou.mybatisplus.annotation.TableId;
import com.baomidou.mybatisplus.annotation.TableName;
import lombok.Data;

import java.time.LocalDateTime;
import java.util.List;

/**
 * 图边定义
 */
@TableName(value = "graph_edge_def", excludeProperty = {"properties"})
@Data
public class GraphEdgeDef extends GraphEntityDef {
    @TableId(type = IdType.AUTO)
    private Long id;

    /**
     * 图 ID
     */
    private Long graphId;

    /**
     * 标签
     */
    private String label;
    /**
     * 边类型名称
     */
    private String name;

    /**
     * 起点类型
     */
    @TableField("start_label")
    private String startLabel;

    /**
     * 终点类型
     */
    @TableField("end_label")
    private String endLabel;
    private String description;

    /**
     * 状态：0-未发布，1-已发布
     */
    private Integer status = 1;
    private LocalDateTime createTime;
    private LocalDateTime updateTime;
    private List<GraphPropertyDef> properties;

    private Boolean multiple;
}
```